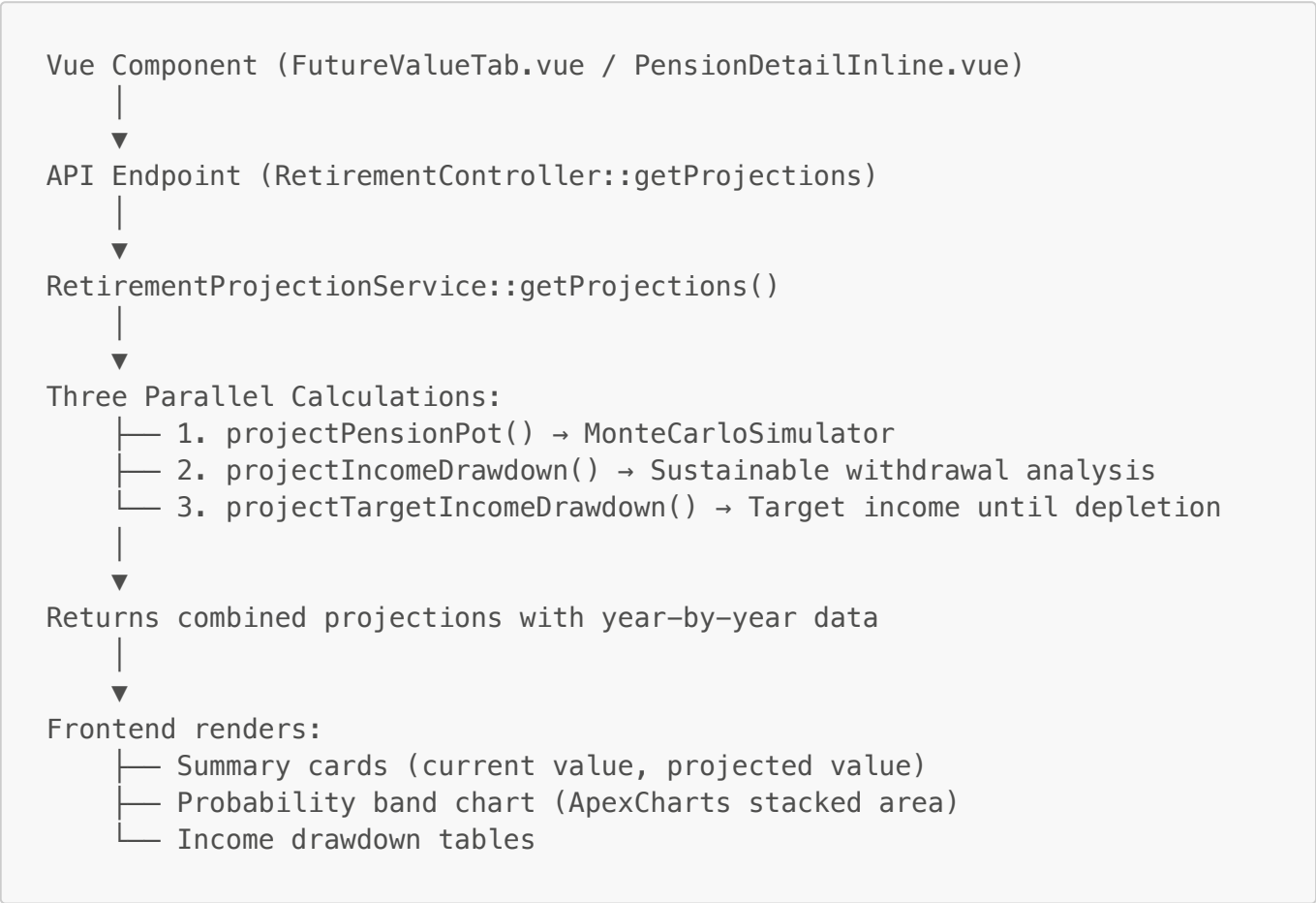


Pension Projection System - Technical Documentation

This document details the complete projection flow, mathematical formulas, Monte Carlo simulation implementation, and design rationale for the pension projection system.

1. Projection Flow (End-to-End)

Architecture Overview



Complete Data Flow

- User Action:** Views Retirement module → Future Value tab or Pension detail → Projections tab
- API Call:** `GET /api/retirement/projections` or `GET /api/retirement/pensions/dc/{id}/projection`
- Controller (RetirementController):**
 - Gets authenticated user
 - Calls `RetirementProjectionService->getProjections($userId)`
- Projection Service (RetirementProjectionService::getProjections):**
 - Aggregates all DC pensions (current_fund_value + monthly_contribution)
 - Retrieves user's risk profile (or defaults to 'medium')

- Executes three calculation methods

5. Monte Carlo Simulator (`MonteCarloSimulator::simulate`):

- Runs 1,000 iterations
- Each iteration: 12 months/year for N years
- Generates random returns using Box-Muller normal distribution
- Collects year-by-year percentiles

6. Probability Bands Extraction (`extractProbabilityBands`):

- Maps raw percentiles to display percentiles
- Blends early years for smooth transition
- Returns 5th, 10th, 15th, 20th, 50th, 75th, 90th percentiles

7. Income Drawdown (`projectIncomeDrawdown`):

- Uses 5th percentile (95% probability) as starting pot
- Applies 4.7% sustainable withdrawal rate
- Factors in DB pensions and State Pension
- Projects year-by-year until fund depletion or age 100

8. Frontend Rendering:

- `PensionPotProjectionChart.vue` - Stacked area chart with probability bands
- Summary cards showing current vs projected values
- Income drawdown tables and charts

2. The Mathematics

DC Pension Future Value Formula

File: `app/Services/Retirement/PensionProjector.php` (lines 36-57)

The core formula is the **Future Value with Regular Contributions**:

$$FV = PV \times (1 + r)^n + PMT \times [((1 + r)^n - 1) / r]$$

Where:

- **FV** = Future Value at retirement
- **PV** = Present Value (current fund value)
- **r** = Net annual growth rate (gross rate - platform fees)
- **n** = Years to retirement
- **PMT** = Annual contribution amount

Component Breakdown

Part 1: Future Value of Current Fund (Compound Interest)

```
$futureValueOfCurrentFund = $currentValue * pow(1 + $netGrowthRate,
$yearsToRetirement);
```

Example:

- Current value: £50,000
- Growth rate: 5% (0.05)
- Years: 20

$$FV = £50,000 \times (1.05)^{20} = £132,665$$

Part 2: Future Value of Contributions (Ordinary Annuity)

```
if ($netGrowthRate > 0 && $annualContribution > 0) {
    $futureValueOfContributions = $annualContribution *
        ((pow(1 + $netGrowthRate, $yearsToRetirement) - 1) /
$netGrowthRate);
} elseif ($annualContribution > 0) {
    // If growth rate is 0%, use simple multiplication
    $futureValueOfContributions = $annualContribution *
$yearsToRetirement;
}
```

Example:

- Annual contribution: £6,000
- Growth rate: 5%
- Years: 20

$$FV = £6,000 \times [((1.05)^{20} - 1) / 0.05]$$

$$FV = £6,000 \times [(2.6533 - 1) / 0.05]$$

$$FV = £6,000 \times 33.066$$

$$FV = £198,396$$

Part 3: Platform Fee Adjustment

```
$netGrowthRate = $growthRate - ((float) $pension->platform_fee_percent ??
0.0) / 100;
```

Example:

- Gross growth: 5.0%
- Platform fee: 0.25%
- Net growth: 4.75%

Annual Contribution Calculation

File: `app/Services/Retirement/PensionProjector.php` (lines 250-272)

Two methods with priority order:

```
private function calculateAnnualContribution(DCPension $pension): float
{
    // Priority 1: Fixed monthly contribution (SIPP/personal pensions)
    $monthlyContribution = (float) ($pension->monthly_contribution_amount
?? 0.0);
    if ($monthlyContribution > 0) {
        return $monthlyContribution * 12;
    }

    // Priority 2: Percentage-based contributions (workplace pensions)
    $annualSalary = (float) ($pension->annual_salary ?? 0.0);
    if ($annualSalary > 0) {
        $employeePercent = (float) ($pension-
>employee_contribution_percent ?? 0.0);
        $employerPercent = (float) ($pension-
>employer_contribution_percent ?? 0.0);
        $totalPercent = $employeePercent + $employerPercent;

        if ($totalPercent > 0) {
            return $annualSalary * ($totalPercent / 100);
        }
    }

    return 0.0;
}
```

Example (Workplace Pension):

- Salary: £62,000
- Employee: 3%
- Employer: 3%
- Total: 6%
- Annual contribution: £62,000 × 0.06 = £3,720

Growth Rates by Risk Level

File: `app/Services/Risk/RiskPreferenceService.php` (lines 34-85)

Risk Level	Min Return	Typical Return	Max Return	Volatility
------------	------------	----------------	------------	------------

Risk Level	Min Return	Typical Return	Max Return	Volatility
Low	1.0%	2.0%	3.0%	3.0%
Lower-Medium	2.0%	3.5%	4.5%	6.0%
Medium	3.5%	5.0%	6.5%	10.0%
Upper-Medium	5.0%	6.5%	8.5%	15.0%
High	6.0%	8.0%	12.0%	20.0%

Risk Level Priority

File: `app/Services/Retirement/PensionProjector.php` (lines 202-211)

```
For each DC Pension:
  1. IF pension has custom risk (has_custom_risk=true)
    → Use pension.risk_preference
  2. ELSE IF user has risk profile in Risk module
    → Use user's risk level
  3. ELSE
    → Default to 'medium' (5% growth, 10% volatility)
```

Sustainable Withdrawal Rate

File: `app/Services/Retirement/RetirementProjectionService.php` (line 22)

```
private const SUSTAINABLE_WITHDRAWAL_RATE = 0.047; // 4.7%
```

Annual income calculation:

```
Annual_DC_Drawdown = Remaining_Fund × 0.047
```

Example:

- Pension pot at retirement: £500,000
- Annual drawdown: £500,000 × 0.047 = £23,500

3. Monte Carlo Simulation

Why Monte Carlo?

Deterministic projections show a single outcome:

```
"You'll have £500,000 at retirement"
```

Monte Carlo projections show a range of outcomes with probabilities:

"You have a 95% chance of having at least £400,000, and could have up to £650,000"

Key Benefits

1. Captures Market Volatility

- Real markets don't grow smoothly at 5% per year
- Monte Carlo simulates realistic market fluctuations

2. Sequence of Returns Risk

- A down market in year 1 vs year 20 has different impacts
- Monte Carlo tests many different orderings

3. Probability-Based Communication

- Users understand "95% chance" better than single estimates
- Builds appropriate expectations

4. Stress Testing

- 1,000 iterations include many worst-case scenarios
- Users see downside risk, not just average

Implementation

File: `app/Services/Investment/MonteCarloSimulator.php` (lines 20-94)

```
public function simulate(
    float $startValue,
    float $monthlyContribution,
    float $expectedReturn,      // Annual return (e.g., 0.05 for 5%)
    float $volatility,          // Annual volatility (e.g., 0.10 for 10%)
    int $years,
    int $iterations = 1000
): array {
    // Convert annual to monthly parameters
    $monthlyReturn = $expectedReturn / 12;
    $monthlyVolatility = $volatility / sqrt(12); // See note below
    $totalMonths = $years * 12;

    $results = [];

    for ($i = 0; $i < $iterations; $i++) {
        $portfolioValue = $startValue;
        $yearlyValues = [];

        for ($month = 1; $month <= $totalMonths; $month++) {
            // Generate random return from normal distribution
            $randomReturn = $this->generateNormalDistribution(
                $monthlyReturn,
```

```

        $monthlyVolatility
    );

    // Apply return and add contribution
    $portfolioValue = $portfolioValue * (1 + $randomReturn)
        + $monthlyContribution;

    // Store yearly snapshots
    if ($month % 12 == 0) {
        $yearlyValues[] = $portfolioValue;
    }
}

$results[] = [
    'final_value' => $portfolioValue,
    'yearly_values' => $yearlyValues
];

return $this->calculatePercentiles($results);
}

```

Why Divide Volatility by $\sqrt{12}$?

```
$monthlyVolatility = $volatility / sqrt(12);
```

In finance, standard deviation scales by the **square root of time**:

- Annual volatility: 10%
- Monthly volatility: $10\% / \sqrt{12} = 2.887\%$

This ensures proper scaling when converting annual parameters to monthly simulation steps.

Random Number Generation (Box-Muller Transform)

File: `app/Services/Investment/MonteCarloSimulator.php` (lines 103-118)

```

public function generateNormalDistribution(float $mean, float $stdDev):
float
{
    // Generate two independent uniform random numbers (0 to 1)
    $u1 = mt_rand() / mt_getrandmax();
    $u2 = mt_rand() / mt_getrandmax();

    // Avoid log(0)
    $u1 = max($u1, 1e-10);

    // Box-Muller transform produces normally distributed value
    $z0 = sqrt(-2.0 * log($u1)) * cos(2.0 * M_PI * $u2);

```

```

    // Scale to desired mean and standard deviation
    return $mean + ($z0 * $stdDev);
}

```

Why Box-Muller?

- Transforms uniform random numbers to normal distribution
- Financial market returns are approximately normally distributed
- More accurate than simple averaging methods

Percentile Calculation

File: `app/Services/Investment/MonteCarloSimulator.php` (lines 125-155)

```

private function calculatePercentiles(array $results): array
{
    // Sort all final values
    $finalValues = array_column($results, 'final_value');
    sort($finalValues);

    $count = count($finalValues);

    // Calculate percentiles
    $percentiles = [5, 10, 15, 20, 25, 50, 75, 90, 95];
    $result = [];

    foreach ($percentiles as $p) {
        $index = ceil(($p / 100) * $count) - 1;
        $index = max(0, min($index, $count - 1));
        $result["percentile_{$p}"] = $finalValues[$index];
    }

    return $result;
}

```

Probability Band Extraction

File: `app/Services/Retirement/RetirementProjectionService.php` (lines 385-455)

The simulator produces: 10th, 25th, 50th, 75th, 90th percentiles

Frontend needs: 5th, 10th, 15th, 20th, 50th, 75th, 90th percentiles

Interpolation for missing percentiles:

```

// Get raw percentiles from simulation
$p10 = $yearData['percentile_10'];
$p25 = $yearData['percentile_25'];

```



```
// Calculate spread between bands
$spread = $p25 - $p10;

// Linear interpolation
$p5 = $p10 - ($spread * 0.33);    // Extrapolate below 10th
$p15 = $p10 + ($spread * 0.33);   // Between 10th and 25th
$p20 = $p10 + ($spread * 0.67);   // Between 10th and 25th
```

Early Year Blending (Smooth Transition)

Problem: Monte Carlo has high variance in year 1, creating unrealistic charts.

Solution: Blend Monte Carlo with known starting value for early years.

```
$blendFactor = match($yearIndex) {
  0 => 0.0,    // Year 0: 100% current value
  1 => 0.7,    // Year 1: 70% Monte Carlo + 30% start value
  2 => 0.9,    // Year 2: 90% Monte Carlo + 10% start value
  default => 1.0 // Year 3+: 100% Monte Carlo
};

$finalValue = ($monteCarloValue * $blendFactor)
              + ($startValue * (1 - $blendFactor));
```

Result: Smooth, realistic curves anchored to known starting point.

4. Probability Band Interpretation

What the Bands Mean

Band	Percentile	Label	Interpretation
Darkest (bottom)	5th	95% Probability	95% chance of achieving this or higher
	10th	90% Probability	90% chance of achieving this or higher
	15th	85% Probability	85% chance of achieving this or higher
	20th	80% Probability	80% chance of achieving this or higher
Lightest (top)	50th	Median	50% chance above, 50% below

Example Interpretation

If the chart shows at retirement:

- 95% band (5th percentile): £400,000
- Median (50th percentile): £550,000
- 80% band (20th percentile): £480,000

This means:

- **95% chance** pension pot will be **at least £400,000**
- **50% chance** it will be above £550,000
- Only **5% chance** it will be below £400,000

5. Monte Carlo vs Deterministic: Design Rationale

What Monte Carlo Overrides

Aspect	Deterministic	Monte Carlo
Growth	Fixed 5%/year	Random, averaging 5% with variance
Output	Single value	Range with probabilities
Worst case	Not shown	Built into lower bands
User understanding	"You'll have X"	"95% chance of at least X"

Why Monte Carlo Was Chosen

1. **More Realistic:** Markets don't grow at exactly 5% per year
2. **Risk Communication:** Shows downside, not just average
3. **Sequence Risk:** Captures impact of return ordering
4. **Retirement Safety:** "Will I run out of money?" needs probability answer
5. **User Trust:** Probability bands feel more honest than point estimates

When Deterministic Is Still Used

- **Scenario calculations:** Quick what-if comparisons use 5% fixed rate
- **Strategy impact:** Additional contribution projections use simplified math
- **Performance:** Full Monte Carlo too slow for real-time slider updates

6. Income Drawdown Projection

Retirement Phase Calculation

File: `app/Services/Retirement/RetirementProjectionService.php` (lines 188-290)

```
public function projectIncomeDrawdown(  
    float $startingPot,           // From 5th percentile at retirement  
    float $targetIncome,  
    int $retirementAge,  
    array $dbPensions,  
    ?StatePension $statePension,  
    string $riskLevel  
): array {  
    $withdrawalRate = 0.047;      // 4.7%  
    $inflationRate = 0.02;        // 2% annual
```

```

    $riskParams = $this->riskService->getReturnParameters($riskLevel);
    $drawdownGrowthRate = $riskParams['expected_return_min'] / 100; //
Conservative!

    $remainingFund = $startingPot;
    $currentAge = $retirementAge;
    $projections = [];

    while ($remainingFund > 0 && $currentAge <= 100) {
        // Calculate DC drawdown
        $dcDrawdown = $remainingFund * $withdrawalRate;

        // Add guaranteed income
        $dbIncome = $this->sumDBPensions($dbPensions);
        $stateIncome = ($currentAge >= $statePensionAge) ?
    $statePensionAnnual : 0;

        $totalIncome = $dcDrawdown + $dbIncome + $stateIncome;

        // Inflate target income
        $inflatedTarget = $targetIncome * pow(1 + $inflationRate,
    $currentAge - $retirementAge);

        // Update fund: grow then withdraw
        $remainingFund = $remainingFund * (1 + $drawdownGrowthRate) -
    $dcDrawdown;

        $projections[] = [
            'age' => $currentAge,
            'dc_drawdown' => $dcDrawdown,
            'db_income' => $dbIncome,
            'state_pension' => $stateIncome,
            'total_income' => $totalIncome,
            'target_income' => $inflatedTarget,
            'remaining_fund' => max(0, $remainingFund),
            'above_target' => $totalIncome >= $inflatedTarget
        ];

        $currentAge++;
    }

    return $projections;
}

```

Why Conservative Growth During Drawdown?

```

$drawdownGrowthRate = $riskParams['expected_return_min'] / 100;

```

Uses **minimum** expected return (not typical):

- Medium risk: uses 3.5% (not 5%)
- During accumulation: Can tolerate volatility
- During drawdown: Must be conservative to avoid running out

7. Retirement Probability Calculation

File: `app/Services/Retirement/RetirementProjectionService.php` (lines 625-660)

```
public function calculateRetirementProbability(
    float $projectedIncome,
    float $targetIncome,
    ?int $fundLastsYears = null
): array {
    // Income coverage ratio
    $incomeRatio = $targetIncome > 0 ? $projectedIncome / $targetIncome :
1.0;

    // Base probability from income coverage
    $probability = match(true) {
        $incomeRatio >= 1.0 => 95,    // 100%+ of target
        $incomeRatio >= 0.90 => 85,   // 90-99%
        $incomeRatio >= 0.75 => 65,   // 75-89%
        $incomeRatio >= 0.50 => 40,   // 50-74%
        $incomeRatio >= 0.25 => 20,   // 25-49%
        default => 10                 // <25%
    };

    // Longevity bonus
    if ($fundLastsYears !== null) {
        if ($fundLastsYears >= 35) {
            $probability = min(95, $probability + 5);
        } elseif ($fundLastsYears >= 25) {
            $probability = min(95, $probability + 3);
        }
    }

    // Status label
    $status = match(true) {
        $probability >= 90 => 'Excellent',
        $probability >= 80 => 'On Track',
        $probability >= 60 => 'Needs Attention',
        $probability >= 35 => 'Off Track',
        $probability >= 15 => 'Significantly Off Track',
        default => 'Critical'
    };

    return [
        'probability' => $probability,
        'status' => $status,
        'income_ratio' => $incomeRatio
    ];
}
```

```
    ];  
  }
```

8. API Endpoints

GET /api/retirement/projections

Full retirement projections including pension pot and income drawdown.

Response:

```
{  
  "pension_pot_projection": {  
    "current_value": 150000,  
    "monthly_contribution": 1000,  
    "risk_level": "medium",  
    "expected_return": 5,  
    "volatility": 10,  
    "years_to_retirement": 20,  
    "retirement_age": 65,  
    "percentile_5_at_retirement": 403456,  
    "median_at_retirement": 549827,  
    "year_by_year": [  
      {  
        "year": 2026,  
        "percentile_5": 150000,  
        "percentile_10": 152000,  
        "percentile_15": 154000,  
        "percentile_20": 156000,  
        "percentile_50": 165000,  
        "percentile_75": 178000,  
        "percentile_90": 195000  
      }  
    ]  
  },  
  "income_drawdown": {  
    "starting_pot": 403456,  
    "target_income": 45000,  
    "retirement_age": 65,  
    "withdrawal_rate": 4.7,  
    "on_track_status": "On Track",  
    "probability": 85,  
    "fund_depletion_age": null,  
    "guaranteed_income": {  
      "db_pensions": 15000,  
      "state_pension": 11502,  
      "total": 26502  
    },  
    "yearly_income": [...]
```

```
}  
}
```

GET /api/retirement/pensions/dc/{id}/projection

Individual DC pension Monte Carlo projection.

Response:

```
{  
  "pension_id": 1,  
  "scheme_name": "TechCorp Pension",  
  "current_value": 45000,  
  "monthly_contribution": 310,  
  "risk_level": "medium",  
  "expected_return": 5,  
  "volatility": 10,  
  "years_to_retirement": 30,  
  "retirement_age": 65,  
  "percentile_5_at_retirement": 209013,  
  "median_at_retirement": 285000,  
  "year_by_year": [...]  
}
```

9. Key Design Decisions

Decision	Value	Rationale
Monte Carlo iterations	1,000	Balance of accuracy vs performance
Withdrawal rate	4.7%	UK pension decumulation standard
Conservative drawdown growth	Min return	Safety margin during retirement
Target income default	75% of net	UK financial planning convention
Inflation rate	2%	Bank of England target
Early year blending	30% →10% →0%	Smooth charts, anchor to known values

10. Key Files Reference

File	Purpose
app/Services/Retirement/PensionProjector.php	Core FV calculations, contribution logic

File	Purpose
app/Services/Retirement/RetirementProjectionService.php	Monte Carlo orchestration, income drawdown
app/Services/Investment/MonteCarloSimulator.php	Monte Carlo implementation
app/Services/Risk/RiskPreferenceService.php	Risk level → return/volatility mapping
app/Http/Controllers/Api/RetirementController.php	API endpoints
resources/js/components/Retirement/PensionPotProjectionChart.vue	Probability band chart
resources/js/components/Retirement/FutureValueTab.vue	Projection display tab